Section 17: Dynamic Programming

This section is part of the 5-hour LeetCode DSA Foundations course by Mukul Raina

- YouTube Course: https://youtu.be/YJMBv4Q2Tg8?si=Q1vDzusUJf753GG3
- YouTube Channel: https://www.youtube.com/@mukulraina
- Instructor LinkedIn: https://www.linkedin.com/in/mukulraina/

C11: DSA Foundations for Interviews

Section 17: Dynamic Programming Foundations

SLIDE 162: Section Intro, Dynamic Programming

Section 17: Dynamic Programming Foundations

Dynamic programming is the technique freshers fear most, but the core concept is simple: recursion with memory. If a recursive solution solves the same subproblem multiple times, store each result the first time and look it up every time after. That single optimization takes exponential algorithms to polynomial.

Section 12 built recursive thinking: base case, recursive case, trust the recursion. Section 13 introduced greedy algorithms and showed where they fail: coin change with denominations [1, 3, 4] and target 6. Greedy picked 4+1+1 = 3 coins. The optimal answer is 3+3 = 2 coins. Section 13 ended with the statement: "When greedy fails and the problem has overlapping subproblems, reach for DP." This section delivers on that promise.

This is foundations-level DP: what DP is, how to convert recursion into DP, the three-part recipe that applies to every DP problem, and four classic problems that appear repeatedly in fresher interviews. Advanced DP patterns (interval DP, DP on trees, bitmask DP, multi-dimensional state design) are covered in C6.

What We'll Cover:

1. When recursion becomes DP: overlapping subproblems and the memoization fix
2. Top-down (memoization) vs bottom-up (tabulation): two routes to the same answer
3. The DP recipe: state definition, transition recurrence, base cases
4. Four worked problems: Climbing Stairs, House Robber, Coin Change, Longest Increasing Subsequence

5. Space optimization: reducing O(n) to O(1) when only recent values are needed

Where DP Connects to Other Sections:

Connection	Section	How It Applies
Recursion is the starting point for every DP solution	Section 12: Recursion	DP = recursion + cache for overlapping subproblems
Greedy fails when local choices are not globally optimal	Section 13: Greedy	DP considers all choices, greedy considers one
Fibonacci memoization was previewed	Section 12: Slide 113	Same technique, now formalized and applied to new problems
Coin change greedy failure was previewed	Section 13: Slide 122	DP solves it correctly where greedy gave the wrong answer
Subsequence problems from Strings section	Section 4: Slide 39	"Subsequence" problems need DP, not substring/sliding window
LIS uses binary search for the O(n log n) optimization	Section 8: Binary Search	Patience sorting variant covered in C6

Mukul Raina | youtube.com/@mukulraina | slides.mukulraina.com | LinkedIn

---

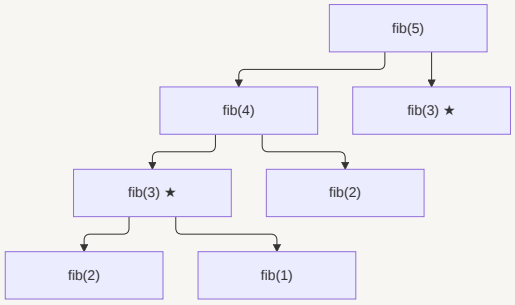
SLIDE 163: When Recursion Becomes DP

The Problem: Overlapping Subproblems

In Section 12 (Slide 113), we saw naive Fibonacci recursion:

```
public int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

This makes two recursive calls per level, producing a binary tree of depth n. Time: O(2^n). The reason it is so slow is not the recursion itself but the duplication. fib(5) calls fib(4) and fib(3). fib(4) calls fib(3) and fib(2). fib(3) is computed twice. fib(2) is computed three times. At fib(40), the same subproblems are recomputed hundreds of millions of times.



The nodes marked with ★ are the same subproblem computed independently in different branches. Every duplicate computation is wasted work.

The Fix: Memoization (Top-Down DP)

Before computing fib(k), check if you have already computed it. If yes, return the stored result. If no, compute it, store it, and return it.

```
public int fib(int n) {
    int[] memo = new int[n + 1];
    Arrays.fill(memo, -1);
    return fibMemo(n, memo);
}

private int fibMemo(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];

    memo[n] = fibMemo(n - 1, memo) + fibMemo(n - 2, memo);
    return memo[n];
}
```

The recursive structure is identical. The only addition is three lines: initialize the cache, check before computing, store after computing. Time drops from O(2^n) to O(n) because each fib(k) is computed exactly once and looked up in O(1) every time after.

The Two Conditions for DP:

Condition	What It Means	Example
Overlapping subproblems	The same subproblem appears multiple times in the recursion tree	fib(3) appears in both fib(5) and fib(4) branches
Optimal substructure	The optimal solution to the problem contains optimal solutions to subproblems	Shortest path through city C uses shortest path to C

If both conditions hold, DP applies. If subproblems do not overlap (like merge sort, where each half is independent), memoization adds nothing because no computation is reused. If optimal substructure is missing, DP cannot combine subproblem solutions into a global optimum.

The Insight That Makes DP Click

DP is not a new algorithm. It is a performance optimization applied to recursion you already know how to write. Write the brute force recursion first (Section 12 skills). Notice overlapping subproblems. Add a cache. You now have a DP solution.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 164: Top-Down (Memoization) vs Bottom-Up (Tabulation)

There are two ways to implement DP. Both produce the same results. They differ in direction and implementation style.

Top-Down: Memoization

Start from the original problem and recurse downward. Cache results as you go. The recursion itself decides which subproblems to solve and in what order.

```
public int fibTopDown(int n) {
    int[] memo = new int[n + 1];
    Arrays.fill(memo, -1);
    return helper(n, memo);
}

private int helper(int n, int[] memo) {
    if (n <= 1) return n;
    if (memo[n] != -1) return memo[n];
    memo[n] = helper(n - 1, memo) + helper(n - 2, memo);
    return memo[n];
}
```

Bottom-Up: Tabulation

Start from the smallest subproblems and build upward iteratively. Fill a table (array) from base cases to the target. No recursion, no call stack.

```
public int fibBottomUp(int n) {
    if (n <= 1) return n;
```

```
    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Comparison:

Property	Top-Down (Memoization)	Bottom-Up (Tabulation)
Direction	Starts from target, recurses to base cases	Starts from base cases, iterates to target
Implementation	Recursive + cache	Iterative + table
Call stack	O(depth), risk of StackOverflow on deep recursion	None
Subproblems solved	Only those needed (lazy)	All subproblems in range (eager)
Ease of writing	Natural, follows recursive thinking from Section 12	Requires understanding the fill order
Space optimization	Harder (call stack complicates it)	Easier (can reduce to O(1) when only recent values needed)
Typical interview use	Start here, easier to derive and explain	Convert to this for optimization

The Recommended Interview Workflow:

1. Write the brute force recursion (Section 12 skills)
2. Add memoization (top-down DP). This is the easiest step and proves correctness.
3. If the interviewer asks for optimization, convert to bottom-up
4. If space matters, optimize the bottom-up table to store only the values you need

Starting with bottom-up directly is harder because you must determine the fill order yourself. Starting with top-down lets the recursion determine the order naturally, and converting to bottom-up afterward is mechanical.

Interview Tip: "I'll start with the recursive solution, then add memoization to handle overlapping subproblems. If you'd like, I can also convert it to bottom-up to eliminate the recursion stack." This sentence tells the interviewer you understand the full spectrum.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 165: The DP Recipe: State, Transition, Base Case

Every DP problem, no matter how complex, requires exactly three decisions. Get these right and the code writes itself. Get the state wrong and no amount of clever coding fixes it.

The Three Decisions:

1. State: What does dp[i] represent?

This is the single hardest part of DP. The state definition must satisfy two requirements: it must capture enough information to make a decision at step i, and the answer to the original problem must be derivable from the dp table.

Examples:

- Climbing Stairs: dp[i] = number of distinct ways to reach step i
- House Robber: dp[i] = maximum money obtainable from the first i houses
- Coin Change: dp[amount] = minimum number of coins to make this amount
- LIS: dp[i] = length of the longest increasing subsequence ending at index i

2. Transition: How does dp[i] relate to smaller subproblems?

This is the recurrence relation. It expresses dp[i] in terms of previously computed dp values. The transition encodes the choices available at step i.

Examples:

- Climbing Stairs: dp[i] = dp[i-1] + dp[i-2] (came from one step back or two steps back)
- House Robber: dp[i] = max(dp[i-1], dp[i-2] + nums[i]) (skip house i, or rob it and add to best from two houses back)
- Coin Change: dp[amount] = min(dp[amount - coin] + 1) for each coin denomination
- LIS: dp[i] = max(dp[j] + 1) for all j < i where nums[j] < nums[i]

3. Base Case: What values are trivially known?

These are the dp entries you can fill without the recurrence because they represent the smallest subproblems.

Examples:

- Climbing Stairs: dp[0] = 1, dp[1] = 1
- House Robber: dp[0] = nums[0], dp[1] = max(nums[0], nums[1])

- Coin Change: dp[0] = 0 (zero coins for amount zero)
- LIS: dp[i] = 1 for all i (every single element is an increasing subsequence of length 1)

The Recipe Applied:

Step	Action	Common Mistake
1. Define state in words	Write "dp[i] represents ..." before any code	Jumping to code without knowing what dp[i] means
2. Write the recurrence	Express dp[i] using dp[smaller values]	Using dp[i] in its own definition (circular)
3. Identify base cases	Find the trivially solvable subproblems	Forgetting edge cases (dp[0], dp[1])
4. Choose direction	Top-down or bottom-up	Starting bottom-up before understanding the recursion
5. Implement and trace	Fill the table for a small input	Not verifying with a manual trace
6. Optimize space (optional)	If dp[i] depends only on recent entries, drop the rest	Optimizing before the solution is correct

Interview Tip: When you start a DP problem, say the state out loud before writing any code. "Let me define dp[i] as the number of ways to reach step i." If the interviewer nods, you have the right state and the rest follows. If the state is wrong, no implementation can save you, and stating it early lets the interviewer redirect you before you waste ten minutes coding the wrong recurrence.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 166: Worked Example: Climbing Stairs

Problem (LeetCode 70): You are climbing a staircase with n steps. Each time you can climb 1 step or 2 steps. How many distinct ways can you reach the top?

Step 1: Understand

n = 1: one way (1 step). n = 2: two ways (1+1 or 2). n = 3: three ways (1+1+1, 1+2, 2+1). The number of ways grows as a Fibonacci-like sequence.

Step 2: Define State

dp[i] = number of distinct ways to reach step i.

Step 3: Write Transition

To reach step i, you either came from step i-1 (one step) or step i-2 (two steps). These are the only two possibilities.

dp[i] = dp[i-1] + dp[i-2]

Step 4: Base Cases

dp[0] = 1 (one way to stand at the ground: do nothing). dp[1] = 1 (one way: take one step).

Step 5: Implement

Top-down:

```
public int climbStairs(int n) {
    int[] memo = new int[n + 1];
    Arrays.fill(memo, -1);
    return helper(n, memo);
}

private int helper(int n, int[] memo) {
    if (n <= 1) return 1;
    if (memo[n] != -1) return memo[n];
    memo[n] = helper(n - 1, memo) + helper(n - 2, memo);
    return memo[n];
}
```

Bottom-up:

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Step 6: Trace through n = 5:

i	dp[i-2]	dp[i-1]	dp[i] = dp[i-1] + dp[i-2]
0			1 (base)
1			1 (base)

i	dp[i-2]	dp[i-1]	dp[i] = dp[i-1] + dp[i-2]
2	1	1	2
3	1	2	3
4	2	3	5
5	3	5	8

Answer: 8 distinct ways to climb 5 stairs.

Step 7: Space Optimization

dp[i] depends only on dp[i-1] and dp[i-2]. We do not need the entire array. Two variables suffice.

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1;
    int prev1 = 1;

    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Time O(n), space O(1). Same answer, minimal memory.

The Deeper Point: Climbing Stairs is Fibonacci with a different name. dp[0]=1, dp[1]=1, dp[i]=dp[i-1]+dp[i-2] is the Fibonacci sequence starting from (1, 1). Recognizing this connection means you can solve Climbing Stairs in seconds during an interview. The follow-up "what if you can climb 1, 2, or 3 steps?" changes the recurrence to dp[i] = dp[i-1] + dp[i-2] + dp[i-3] but the structure is identical.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 167: Worked Example: House Robber

Problem (LeetCode 198): You are a robber planning to rob houses along a street. Each house has a certain amount of money. Adjacent houses have connected security systems: if two adjacent houses are robbed, the alarm triggers. Find the maximum amount you can rob without triggering any alarm.

Step 1: Understand

Input: [2, 7, 9, 3, 1]. You cannot rob house 0 and house 1 (adjacent). You can rob house 0 and house 2 (not adjacent). Maximize total.

Step 2: Define State

dp[i] = maximum money obtainable considering only houses 0 through i.

Step 3: Write Transition

At house i, you have exactly two choices:

- Skip house i: your best is whatever you could get from houses 0 through i-1. dp[i] = dp[i-1].
- Rob house i: you take nums[i], but you cannot have robbed house i-1. Your best from the remaining is dp[i-2]. dp[i] = dp[i-2] + nums[i].

Take the better option: dp[i] = max(dp[i-1], dp[i-2] + nums[i]).

Step 4: Base Cases

dp[0] = nums[0] (only one house, rob it). dp[1] = max(nums[0], nums[1]) (two houses, rob the richer one).

Step 5: Implement

```java
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
    }
    return dp[nums.length - 1];
}
```

Step 6: Trace through [2, 7, 9, 3, 1]:

i	nums[i]	dp[i-2] + nums[i]	dp[i-1]	dp[i] = max	Decision
0	2			2 (base)	Rob
1	7			7 (base: max(2,7))	Rob house 1, skip house 0
2	9	2 + 9 = 11	7	11	Rob house 2 (skip 1, take 0+2)
3	3	7 + 3 = 10	11	11	Skip house 3
4	1	11 + 1 = 12	11	12	Rob house 4

Answer: 12. The optimal selection is houses 0, 2, and 4 (2 + 9 + 1 = 12).

Step 7: Space Optimization

Same pattern as Climbing Stairs: dp[i] depends only on dp[i-1] and dp[i-2].

```java
public int rob(int[] nums) {
    if (nums.length == 1) return nums[0];

    int prev2 = nums[0];
    int prev1 = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        int current = Math.max(prev1, prev2 + nums[i]);
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Time O(n), space O(1).

Why This Problem Matters for Interview Understanding

House Robber is the simplest "skip or take" DP problem. The structure, choose between including the current element or excluding it, reappears in dozens of problems: 0/1 Knapsack, Paint Houses, Delete and Earn, and others covered in C6. Master this transition pattern and you have a template for an entire family of problems.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

SLIDE 168: Worked Example: Coin Change

Problem (LeetCode 322): Given an array of coin denominations and a target amount, find the minimum number of coins needed to make that amount. If it is not possible, return -1.

Why This Problem Is Here

In Section 13 (Slide 122), we saw that greedy fails for non-standard denominations. coins = [1, 3, 4], target = 6. Greedy picks 4 + 1 + 1 = 3 coins. Optimal is 3 + 3 = 2 coins. Greedy chose the locally largest coin at each step, but the globally optimal solution uses smaller coins. DP considers all choices at every step and picks the global optimum.

Step 1: Define State

dp[amount] = minimum number of coins to make exactly this amount.

Step 2: Write Transition

For each amount, try using each coin. If the coin fits (coin <= amount), using it leaves a subproblem of amount - coin.

dp[amount] = min(dp[amount - coin] + 1) for each coin where coin <= amount

The +1 accounts for the coin you just used.

Step 3: Base Cases

dp[0] = 0 (zero coins needed for amount zero). Initialize all other entries to amount + 1 as a sentinel for "impossible." Any value larger than the maximum possible number of coins (which is amount when using only denomination 1) works as the sentinel.

Step 4: Implement

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }
    return dp[amount] > amount ? -1 : dp[amount];
}
```

Step 5: Trace through coins = [1, 3, 4], amount = 6:

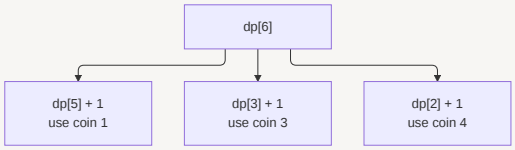
amount	Try coin 1	Try coin 3	Try coin 4	dp[amount]	Coins used
0				0 (base)	
1	dp[0]+1 = 1	3>1, skip	4>1, skip	1	[1]
2	dp[1]+1 = 2	3>2, skip	4>2, skip	2	[1,1]
3	dp[2]+1 = 3	dp[0]+1 = 1	4>3, skip	1	[3]
4	dp[3]+1 = 2	dp[1]+1 = 2	dp[0]+1 = 1	1	[4]
5	dp[4]+1 = 2	dp[2]+1 = 3	dp[1]+1 = 2	2	[1,4] or [3,?]
6	dp[5]+1 = 3	dp[3]+1 = 2	dp[2]+1 = 3	**2**	**[3, 3]**

Answer: 2 coins. dp[6] = 2, achieved by using coin 3 twice. Greedy would have picked coin 4 first and needed 3 coins (4+1+1). DP found the true minimum by considering all options at every amount.

Complexity: Time O(amount * number of coins). Space O(amount). For coins = [1, 3, 4] and amount = 6: 6 * 3 = 18 operations. Compare to greedy's 3 operations, but greedy gave the wrong answer. Correctness is worth the extra computation.

The Pattern: Minimization Over Choices

The transition dp[amount] = min(dp[amount - coin] + 1) for each coin is the minimization-over-choices pattern. At each step, you try every available choice and take the minimum. This same structure appears in Minimum Path Sum, Edit Distance, and other problems in C6.



dp[6] considers three subproblems. dp[3] + 1 = 2 wins. dp[3] itself was computed as dp[0] + 1 = 1 (use coin 3). Total: two coins of value 3.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 169: Worked Example: Longest Increasing Subsequence

Problem (LeetCode 300): Given an integer array, find the length of the longest strictly increasing subsequence (LIS).

Clarification: Subsequence, Not Subarray

A subsequence can skip elements. [2, 3, 7, 101] is a subsequence of [10, 9, 2, 5, 3, 7, 101, 18]. It maintains relative order but does not require contiguity. This is why sliding window (Section 7) does not apply: the elements are not required to be adjacent. As noted in Section 4 (Slide 39), subsequence problems typically require DP.

Step 1: Define State

dp[i] = length of the longest increasing subsequence ending at index i.

The key phrase is "ending at index i." Each dp[i] considers only subsequences that include nums[i] as the last element. The overall answer is the maximum across all dp[i].

Step 2: Write Transition

For each index i, look at every earlier index j where nums[j] < nums[i]. If nums[j] < nums[i], then nums[i] can extend the subsequence ending at j.

dp[i] = max(dp[j] + 1) for all j < i where nums[j] < nums[i]

Step 3: Base Cases

Every single element is an increasing subsequence of length 1. dp[i] = 1 for all i.

Step 4: Implement

```
public int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);

    int maxLen = 1;

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        maxLen = Math.max(maxLen, dp[i]);
    }
    return maxLen;
}
```

Step 5: Trace through [10, 9, 2, 5, 3, 7, 101, 18]:

i	nums[i]	Check all j < i where nums[j] < nums[i]	dp[i]	LIS ending here
0	10	(none)	1	[10]
1	9	No j satisfies nums[j] < 9 (10 > 9)	1	[9]
2	2	No j satisfies nums[j] < 2	1	[2]
3	5	j=2: nums[2]=2 < 5, dp[2]+1 = 2	2	[2, 5]
4	3	j=2: nums[2]=2 < 3, dp[2]+1 = 2	2	[2, 3]
5	7	j=2: 2<7 (dp=2), j=3: 5<7 (dp=3), j=4: 3<7 (dp=3)	3	[2, 5, 7] or [2, 3, 7]
6	101	j=0..5 all < 101, best: dp[5]+1 = 4	4	[2, 5, 7, 101]
7	18	j=2: 2<18 (dp=2), j=3: 5<18 (dp=3), j=4: 3<18 (dp=3), j=5: 7<18 (dp=4)	4	[2, 3, 7, 18]

Answer: max(dp) = 4. One such LIS: [2, 5, 7, 101] or [2, 3, 7, 101].

Complexity: Time O(n^2) due to the nested loop. Space O(n) for the dp array.

Note on Optimization: An O(n log n) solution exists using patience sorting with binary search (Section 8). The idea: maintain a list of the smallest tail elements for increasing subsequences of each length, and use binary search to find the insertion point. This is covered in C6, not here, because the O(n^2) solution is more intuitive and sufficient for most fresher interviews.

The Pattern: Maximize Over All Valid Predecessors

The transition dp[i] = max(dp[j] + 1) for valid j is the maximize-over-predecessors pattern. It differs from the previous problems where the choices were fixed (climb 1 or 2, skip or take, use coin X). Here, the number of valid predecessors varies for each position. The O(n^2) comes from checking every j < i.

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 170: Wrong vs Right: DP Mistakes

Mistake 1: Jumping Straight to Bottom-Up Without Understanding the Recursion

Wrong: trying to write the dp table iteratively without first understanding the recursive structure. Getting confused about fill order, dependencies, and what dp[i] even represents.

Right: write the brute force recursion first. Verify it is correct on a small input. Then add memoization. Once the top-down version works, converting to bottom-up is mechanical:

replace recursion with a loop, replace cache lookups with table reads, fill in order from base case to target.

Mistake 2: Wrong State Definition

Wrong: for House Robber, defining dp[i] as "the maximum if you rob house i." This forces you to always rob house i, which is not always optimal.

Right: dp[i] = "maximum money from houses 0 through i" (whether or not you rob house i). The choice of robbing or skipping lives inside the transition, not the state.

If the state definition is wrong, the transition will not make sense, and no amount of debugging fixes it. When your DP solution produces wrong answers, check the state definition first.

Mistake 3: Forgetting Base Cases

Wrong:

```
int[] dp = new int[n + 1];
for (int i = 2; i <= n; i++) {
    dp[i] = dp[i - 1] + dp[i - 2];
}
```

dp[0] and dp[1] default to 0. For Climbing Stairs, dp[0] should be 1 and dp[1] should be 1. The loop reads uninitialized (or wrong) base values and every subsequent entry is wrong.

Right: explicitly set every base case before the loop begins.

Mistake 4: Not Optimizing Space When Only Recent Values Are Needed

Wrong: using an O(n) array when the transition only reads dp[i-1] and dp[i-2]. The extra memory is wasted.

Right: recognize when the transition has a fixed lookback window. If dp[i] depends only on the last k values, you need O(k) space, not O(n). For Climbing Stairs and House Robber, k = 2. For Coin Change, dp[amount] depends on dp[amount - coin] for all coins, which can look back far, so the full array is needed.

Mistake 5: Using DP When Greedy Works

Wrong: writing a full DP solution for a problem that greedy solves correctly (interval scheduling, Jump Game). DP is correct but overengineered.

Right: try greedy first. Test with a counterexample. If greedy works, use it (simpler, often faster). If you find a counterexample where greedy fails, switch to DP and say so: "I tested greedy with [example] and it gives the wrong answer, so I'll use DP instead." That explicit reasoning is one of the strongest moves in a DP interview.

Summary of DP Debugging Checklist:

Symptom	Likely Cause	Fix
All dp values are 0 or wrong	Base cases not set correctly	Verify dp[0], dp[1] initialization
dp[i] is always the same as dp[i-1]	Transition does not use current element	Check the recurrence: does it reference nums[i]?
Correct for small inputs, wrong for large	Off-by-one in loop bounds	Verify loop starts at the right index and ends at the right index
Solution works but TLE	Using recursion without memoization	Add memo array or convert to bottom-up
Wrong answer on certain inputs	State definition is wrong	Re-examine what dp[i] represents in words

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 171: How to Present DP Solutions in an Interview

The Four-Phase Presentation

Phase 1: Start with Brute Force Recursion

"The brute force approach tries all possible combinations. For Climbing Stairs, at each step I can go up 1 or 2, so I recursively explore both options. This gives a recursion tree of depth n with branching factor 2, so time is O(2^n)."

This shows the interviewer you understand the problem's structure before optimizing.

Phase 2: Identify Overlapping Subproblems, Add Memoization

"I notice that many subproblems are computed multiple times. For example, the number of ways to reach step 3 is needed when computing step 4 and step 5. I'll add a cache so each subproblem is solved only once. This brings the time down to O(n)."

This shows the interviewer you understand why DP applies, not just how to code it.

Phase 3: State the Three Components Explicitly

"My state: dp[i] represents the number of ways to reach step i. The transition: dp[i] = dp[i-1] + dp[i-2], because the last move was either 1 step or 2 steps. Base cases: dp[0] = 1 and dp[1] = 1."

Stating these three components is the single most important thing you can do in a DP interview. It proves structured thinking and makes your code predictable.

Phase 4: Offer Optimizations

"I can convert this to bottom-up to avoid recursion stack overhead. And since dp[i] only depends on dp[i-1] and dp[i-2], I can reduce space from O(n) to O(1) by keeping just two variables."

Offer this proactively. Do not wait for the interviewer to ask. It demonstrates depth.

Handling Common Interviewer Follow-ups:

Follow-up	What They Want	How to Respond
"Can you optimize the space?"	O(1) if transition has fixed lookback	"dp[i] only needs the last two values, so I replace the array with two variables"
"What if you can climb 1, 2, or 3 steps?"	Extend the recurrence	"I add dp[i-3] to the transition and a third base case"
"What is the time and space?"	Both, stated unprompted	"Time is O(n), space is O(1) after optimization"
"Is there a mathematical formula?"	Fibonacci closed form	"For Climbing Stairs specifically, this is the Fibonacci sequence, which has a closed-form expression, but the DP approach is more general and what I would use in practice"

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 172: Interview Practice: Dynamic Programming

Practice Prompt: "You are climbing a staircase with n steps. Each time you can climb 1 or 2 steps. How many distinct ways can you reach the top?"

How to Walk Through This in an Interview:

Step 1: Understand (30 seconds)

"Let me make sure I understand. I have n steps. At each point I can take 1 step or 2 steps. I need the total number of distinct paths to reach step n. For n = 3, the paths are: 1+1+1, 1+2, 2+1. That's 3 ways."

Step 2: Brute Force (30 seconds)

"The brute force is to recursively try both options at each step. This gives a binary recursion tree of depth n, so time is O(2^n). That's too slow for large n."

Step 3: Recognize DP (30 seconds)

"I notice overlapping subproblems. The number of ways to reach step 3 is needed when computing step 4 and step 5. So I'll use dynamic programming."

Step 4: State the Three Components (30 seconds)

"dp[i] represents the number of distinct ways to reach step i. The transition is dp[i] = dp[i-1] + dp[i-2]. Base cases: dp[0] = 1, dp[1] = 1."

Step 5: Implement (2 minutes)

```
public int climbStairs(int n) {
    if (n <= 1) return 1;

    int prev2 = 1;
    int prev1 = 1;

    for (int i = 2; i <= n; i++) {
        int current = prev1 + prev2;
        prev2 = prev1;
        prev1 = current;
    }
    return prev1;
}
```

Step 6: Verify (30 seconds)

"For n = 5: dp values are 1, 1, 2, 3, 5, 8. The answer is 8. Time O(n), space O(1) since I only track the last two values."

Common Interviewer Follow-ups and Expected Responses:

Follow-up	Expected Response
"What if you can climb 1, 2, or 3 steps?"	"Transition becomes dp[i] = dp[i-1] + dp[i-2] + dp[i-3]. I track three variables instead of two. Still O(n) time, O(1) space."
"What if certain steps are broken and cannot be used?"	"I add a condition: if step i is broken, dp[i] = 0 (no way to stand on it). The transition for non-broken steps stays the same."
"Can you do this recursively with memoization?"	"Yes." (Write the top-down version with memo array.)

Common Fresher Mistakes:

Mistake	Why It Fails	Fix
Writing brute force and stopping	O(2^n) is too slow for large n	Recognize overlapping subproblems, add DP
Jumping to bottom-up directly	Harder to derive, easier to have wrong fill order	Start recursive, add memo, then convert

Mistake	Why It Fails	Fix
Not stating dp[i] definition	Interviewer cannot follow your logic	Say "dp[i] represents ..." before any code
Forgetting n=0 and n=1 edge cases	Crash or wrong answer on small inputs	Handle base cases explicitly before the loop
Not mentioning space optimization	Missed opportunity to show depth	Proactively optimize from O(n) to O(1)

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina

---

SLIDE 173: Section Cheat Sheet

The DP Recipe:

1. Define state: what does dp[i] represent? (Write this in words before any code.)
2. Write the transition: how does dp[i] relate to smaller dp values?
3. Identify base cases: what are the trivially known values?
4. Choose direction: top-down (memoization, easier to derive) or bottom-up (tabulation, easier to optimize)
5. Implement and trace on a small input
6. Optimize space if dp[i] depends only on recent entries

Top-Down vs Bottom-Up:

Property	Top-Down	Bottom-Up
Style	Recursive + cache	Iterative + table
Direction	Target to base cases	Base cases to target
Strength	Natural to write, only solves needed subproblems	No call stack, easier space optimization
Start here for	Deriving the solution, explaining to interviewer	Final optimized version

Classic Problems Quick Reference:

Problem	State	Transition	Time	Space
Climbing Stairs	dp[i] = ways to step i	dp[i-1] + dp[i-2]	O(n)	O(1)
House Robber	dp[i] = max from first i houses	max(dp[i-1], dp[i-2]+nums[i])	O(n)	O(1)

Problem	State	Transition	Time	Space
Coin Change	dp[amt] = min coins for amt	min(dp[amt-coin]+1) per coin	O(amt*coins)	O(amt)
LIS	dp[i] = LIS ending at i	max(dp[j]+1) for valid j	O(n^2)	O(n)

DP Pattern Categories:

Category	Signal	Structure	Examples
Counting	"How many ways"	Sum choices: dp[i] = sum of dp[smaller]	Climbing Stairs, Unique Paths
Optimization	"Minimum/maximum"	Best over choices: dp[i] = min/max over options	Coin Change, House Robber
Feasibility	"Can you reach/make"	Boolean OR: dp[i] = any(dp[smaller])	Word Break, Partition Equal Subset
Subsequence	"Longest/shortest subsequence"	Check all valid predecessors	LIS, Longest Common Subsequence (C6)

Pattern Recognition Signals for DP:

Signal	Why DP	Instead of
Overlapping subproblems in recursion tree	Same subproblem solved many times	Plain recursion (exponential)
Greedy counterexample found	Local optimum is not global optimum	Greedy (wrong answer)
"Subsequence" (not substring)	Non-contiguous, need to consider all options	Sliding window (only works for contiguous)
"Number of ways to..."	Counting problem with choices at each step	Backtracking (generates all, then counts; DP counts directly)
"Minimum/maximum cost/profit" + choices	Optimization with multiple valid paths	Brute force (tries all, exponential)

Space Optimization Rule:

If dp[i] depends only on dp[i-1] and dp[i-2], replace the array with two variables (O(1) space). If dp[i] depends on dp[i-k] for fixed k, use a k-size rolling window. If dp[i] depends on any dp[j] for j < i (like LIS or Coin Change), the full array is needed.

How This Connects to Other Sections:

This Section's Concept	Connects To	How
Memoized recursion	Section 12: Recursion	DP starts from the brute force recursion you already wrote

This Section's Concept	Connects To	How
Greedy failure as DP trigger	Section 13: Greedy	"Greedy gave wrong answer, switching to DP" is a strong interview statement
Fibonacci recurrence	Section 12: Slide 113	Climbing Stairs is Fibonacci with different base case labels
Coin Change correctness	Section 13: Slide 122	DP produces correct answer where greedy failed
Subsequence vs substring	Section 4: Slide 39, Section 7	Substring uses sliding window, subsequence uses DP
O(n log n) LIS optimization	Section 8: Binary Search	Patience sorting uses binary search on tail elements (C6)
Advanced DP patterns	C6: DP	Interval DP, DP on trees, bitmask DP, multi-dimensional state

Mukul Raina | youtube.com/@mukulraina | linkedin.com/in/mukulraina